**Imperial College London**

Department of Earth Science and Engineering

MSc in Environmental Data Science and Machine Learning

**Independent Research Project**

**Final Report**

# ClimateTech Startup-Investor Matching Tool

by

**Lingxin Kong**

**GitHub IRP repository:**

https://github.com/ese-ada-lovelace-2025/irp-lk125

**Supervisors and significant support:**

Maiko Schaffrath

César A. Quilodrán-Casas

Chuan Ju

Submission month and year: August 2026

# Acknowledgements

I would like to thank my supervisors, Maiko Schaffrath and César A. Quilodrán-Casas, for their guidance throughout the project. They kept asking me to be clear about what the code showed and what the evaluation could actually support, which changed the way I approached the final report. I am also grateful to Chuan Ju for her work on the qualification procedure, the evidence checks and the final receiver-side verification. Any mistakes that remain are my own.

# Abstract

Early-stage ClimateTech founders do not need a long list of investors who might be eligible. They need a short list in an order that makes sense. I kept Undaunted's existing hybrid matcher as the legacy baseline and tested one focused change, normalisation-v1. It standardises common stage, geography and instrument aliases. For experiments, it also applies a stable investor-ID tie-break and keeps deterministic text similarity separate from the production GPT route. I built a protected runner around the matcher to freeze inputs and configuration, create algorithm-blinded review files and check returned evidence before calculating metrics. The final comparison used four anonymised funding rounds and a verified historical snapshot of 383 investors. One model-assisted qualification pass labelled 56 pooled case-investor pairs. Macro P@5 rose from 0.350 to 0.500, and macro P@10 from 0.300 to 0.400. Cases 1 and 2 improved; Cases 3 and 4 did not change. The results are descriptive, not a population estimate or human expert validation. They show a traceable improvement under the tested conditions and provide a process for a larger human review.

# Contents

**Acknowledgements** ..... **2**
**Abstract** ..... **3**
**1. Introduction** ..... **5**
1.1 Practical Setting ..... 5
1.2 Research Question and Scope ..... 5
1.3 Objectives and Contribution ..... 5
**2. Background and Baseline System** ..... **6**
2.1 Investor Matching as Ranked Retrieval ..... 6
2.2 Rule-based and Semantic Matching ..... 6
2.3 Preserved Undaunted Baseline ..... 6
**3. Optimisation Design and Implementation** ..... **8**
3.1 Controlled Engineering Principles ..... 8
3.2 Canonical Normalisation ..... 8
3.3 Ranking and Semantic Boundaries ..... 8
3.4 Reproducibility and Integration ..... 9
**4. Evaluation Methodology** ..... **10**
4.1 Study Design and Frozen Inputs ..... 10
4.2 Compared Profiles and Controls ..... 10
4.3 Pooled Algorithm-blinded Qualification ..... 10
4.4 Metrics, Integrity and Reporting Boundary ..... 11
**5. Results** ..... **13**
5.1 Implementation and Evidence Checks ..... 13
5.2 Headline Comparison ..... 13
5.3 Per-case Behaviour ..... 14
5.4 Supplementary Pilot and Platform Provenance ..... 14
**6. Discussion and Limitations** ..... **15**
6.1 Answer to the Research Question ..... 15
6.2 What Worked and What was Difficult ..... 15
6.3 Limitations and Threats to Validity ..... 15
6.4 Privacy, Ethics and Deployment Boundary ..... 15
**7. Conclusion and Future Work** ..... **17**
**Research Declarations** ..... **18**
**References** ..... **19**

# 1. Introduction

## 1.1 Practical Setting

Early-stage ClimateTech founders can find plenty of investor names. The harder part is deciding who to approach first. A fund may fit one field and still be a poor match because its cheque size, stage, geography or investment structure does not fit the round. Fifty plausible names are less useful than a short list in an order that can be explained. I treat this as a ranking problem: the funding round is the query, investors are the candidates, and the strongest matches should appear near the top.

Undaunted already had a working startup-investor tool built around a web form, Airtable records, weighted rules and a GPT semantic stage. I did not need another interface to demonstrate the idea. The task was to improve the ranking without breaking the workflow already in use. Better ordering could reduce weak outreach and make each shortlist easier to check. It still cannot show whether an investor will agree to a meeting or invest. My aim is narrower: improve qualification precision near the top of the list.

## 1.2 Research Question and Scope

The baseline combined field-level rules with semantic reranking, but there was no controlled way to compare a new version with the old one. Airtable order, stored feedback and the external model could all change between runs. A different list was easy to produce, but it was hard to explain where the difference came from. I therefore asked: does the final optimised matcher return more appropriate Top-10 investor matches than the preserved legacy matcher when both receive the same complete ClimateTech funding-round inputs and the same investor pool? This question concerns ranked-list relevance only. It does not make a claim about introductions, meetings or investment outcomes.

## 1.3 Objectives and Contribution

The project had three practical objectives. First, preserve and document the old matcher so later comparisons used a fixed baseline. Second, change label handling and ranking without changing the frontend result shape. Third, build a private evaluation process that freezes inputs, hides the profile name during qualification and checks returned evidence before calculating P@5 and P@10. Success meant that both profiles ran on identical inputs, kept the same output contract and produced a package that passed the integrity checks. I considered supervised learning-to-rank, but the project did not have the large, stable label set it needs (Liu, 2009). The contribution is a small engineering change that can be isolated, together with the evidence trail used to judge it.

# 2. Background and Baseline System

## 2.1 Investor Matching as Ranked Retrieval

An investor database can contain many funds that are technically eligible for a round. Eligibility on its own is not enough. A founder needs a short list in a useful order. I model investor matching as ranked retrieval: the funding round is the query, each investor is a candidate, and the score is an estimate of relevance. This follows the information-retrieval setting described by Manning, Raghavan and Schütze (2008), where order matters because users rarely inspect every result.

The difference between filtering and ranking also affects evaluation. Ten eligible investors can still make a poor shortlist if the best matches sit near the bottom. I use precision at five and ten because the labels are binary and the practical question is whether an investor belongs in the founder's shortlist. Mean reciprocal rank and normalised discounted cumulative gain would reward relevant items near the top, while NDCG can also use graded relevance (Järvelin and Kekäläinen, 2002). With only four complete cases, Top-K measures are easier to interpret and do not suggest more precision than the study can support.

## 2.2 Rule-based and Semantic Matching

Rule-based matching is a sensible starting point because every part of the score can be traced to a field. Stage, ticket size, sector, geography and investment instrument all describe genuine compatibility constraints. The weakness is label consistency. A person reads 'Pre seed' and 'Pre-seed' as the same stage, but an exact lookup can treat them as different values. Reweighting the score will not repair that mismatch. The labels need to be cleaned before the scoring formula is changed.

Longer text fields contain information that rules handle badly, including investment theses, climate problems and business models. Sentence embeddings can compare this text reproducibly through cosine similarity (Reimers and Gurevych, 2019). A large language model can make a more contextual judgement, but it brings an external API, model-version dependence and variation between runs. In the preserved matcher, semantic scoring is one part of a hybrid score. Learning-to-rank could learn from labelled examples (Burges et al., 2005), but I ruled it out because the project had too few stable relevance labels.

## 2.3 Preserved Undaunted Baseline

The preserved Undaunted path begins when the frontend sends a funding-round form to POST /api/investor-introductions/submit. The backend validates the form, creates a matching job and loads the investor records from Airtable. It also reads stored feedback and the active configuration before building the fundingRound and startup objects passed to runMatching. Airtable provides stage, sector, regional focus, ticket range, instruments, description and investment thesis. When matching finishes, the route saves the results and returns the same object shape expected by the interface.

runMatching calculates five rule scores for every investor. Stage and sector each account for 20% of the weighted total, followed by ticket size at 15%, geography at 10% and instruments at 5%. Semantic scoring supplies the remaining 30%. Preferred investor type is handled separately through a multiplier of 1.10 for a match and 0.90 for a non-match. Because that multiplier affects both the rule-based order and the final total, it can change which investors reach semantic scoring and where they finish.

Only the highest rule-ranked candidates are sent to gpt-4o-mini. The default limit is 60 and the requests are processed in batches of five. If topN is supplied, the candidate limit becomes three times

topN, capped at 50. Everyone else stays in the list with a semantic score of zero. For the scored candidates, the model returns a score, an explanation and an exclusion flag; an exclusion caps the final score at 15. The matcher then sorts by totalScore and returns the investor, scoreBreakdown, matchReasons and, when available, aiInsight.

The baseline works in production, but it is difficult to study. Equivalent labels can miss each other, and tied totals fall back to source order because there is no explicit tie-break. The earlier rule ranking decides who receives semantic coverage. Repeating the GPT stage also needs credentials and may not return the same scores. On top of that, Airtable data, feedback and configuration can all change outside the experiment. I therefore focused the optimisation on canonical labels, stable ordering and frozen comparison inputs while leaving the frontend contract alone.

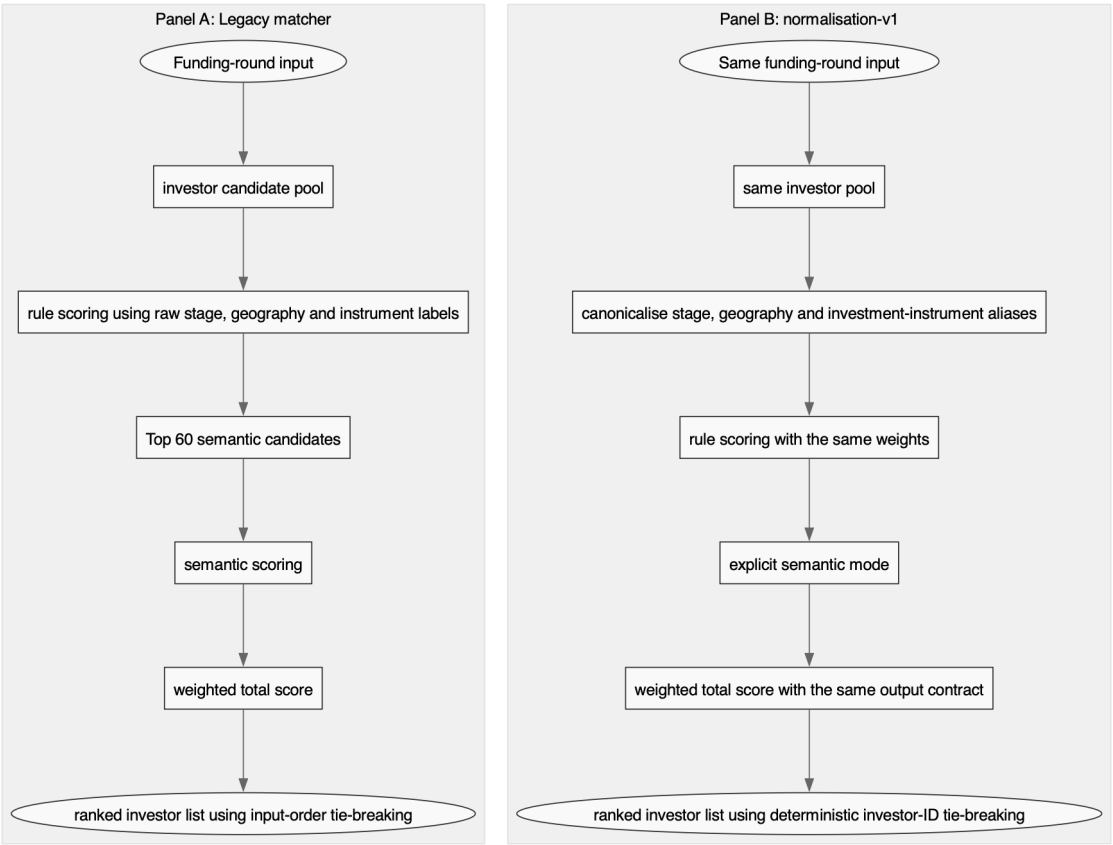



Figure 1. Preserved legacy and normalisation-v1 matcher workflows.

# 3. Optimisation Design and Implementation

## 3.1 Controlled Engineering Principles

I kept the old matcher instead of replacing it. The implementation has two named profiles, legacy and normalisation-v1. Legacy follows the original scoring path with the same weights and behaviour, so it remains a usable baseline. normalisation-v1 keeps that scoring structure but cleans selected labels before the rules run. Giving each path a name made the comparison much easier to follow and reduced the chance that an unrelated edit would be mistaken for an improvement. I chose not to build a broad fuzzy matcher. It might increase some scores, but I would have had far less confidence about which rule caused the change.

The implementation cut-off mattered as much as the code. Labels collected afterwards were used for evaluation only. I did not add them to the alias list or change the weights, because that would let normalisation-v1 adapt to the test cases. I also kept the production prompt and frontend response shape. The change is deliberately small: canonical labels, deterministic experimental ordering and tighter comparison controls. It is less ambitious than replacing the model, but it fits the evidence available. The legacy profile remains runnable, so an output difference has a short path back to its likely cause.

## 3.2 Canonical Normalisation

The normalisation layer runs immediately before rule scoring. It first puts text into a consistent form using Unicode normalisation, lower case, trimmed spaces and common separator handling. It then checks alias maps for funding stage, geography and investment instrument, the three fields where I found inconsistent labels. For example, 'pre seed' becomes 'Pre-seed', 'UK' becomes 'United Kingdom', and 'convertible loan' becomes 'Convertible Note'. The canonical labels use wording that the existing scorer already understands, so I did not need a second set of rules. The code also keeps the change visible: it reads the raw record, creates a canonical copy for matching and then continues through the familiar scoring path.

I added two safeguards to the helper. It removes duplicates after mapping because different aliases can collapse to the same canonical value. Without that check, one concept could appear twice and make debugging needlessly confusing. Unknown labels are handled more cautiously: spacing is cleaned, but the value is otherwise kept. I did not want an unfamiliar term to vanish or be pushed into a category just because it looked similar. A missing alias may therefore leave an item unmatched. That is not ideal, but it is visible and can be reviewed when new labels appear.

## 3.3 Ranking and Semantic Boundaries

In the original matcher, equal totals could leave investors in source-row order. That order came from Airtable or the input file, not from a meaningful ranking decision. For the paired experiments, I used investor ID as an explicit tie-break after total score. Both profiles could then be compared against the same stable order. I left the default production behaviour unchanged because altering it would affect historical behaviour outside the study. Instead, the experiment runner selects the deterministic option. This improves repeatability without implying that the live route has already been replaced.

I split semantic scoring into three explicit modes. Rule-only stops after the seven rule scores. Text-similarity adds a local cosine score based on token counts, needs no API key and returns the same answer for the same text. GPT-semantic keeps the authorised model-based scorer and fails openly if its

credentials or output are invalid. It cannot silently fall back to local text similarity under the same name. The local scorer is useful for repeatable comparisons, but it is not a substitute for GPT. Historical feedback is a separate ablation. A named run can enable it, but the main profile comparison does not.

## 3.4 Reproducibility and Integration

The comparison runner controls the setup rather than changing the score. It rejects missing or repeated investor IDs, invalid weights, bad candidate limits and incomplete inputs. Both profiles receive the same cases and investor pool. Each export stores hashes for the inputs, configuration, matcher file, pool and candidate order, together with the Git commit. Formal runs also stop on a dirty worktree and will not overwrite an existing result file. These checks are simple, but they show that two result files came from the setup described in this report.

The frontend result contract stayed the same. Every match still contains the investor, seven score components, match reasons and the AI insight field. The runtime adapter validates its input and records the model and configuration, while the route still handles saving, completion and notification around the original request object. I built a standalone interface with fictional records so that the real matcher path can be tested without exposing private data or depending on a live account. The guarded runtime is ready to integrate, but I have not switched the production route. That decision should come after review of the frozen evaluation. Section 4 therefore compares the two profiles on exactly the same inputs.

Table 1. Comparison of the preserved legacy and normalisation-v1 matcher profiles.

| Component | Legacy | normalisation-v1 | Rationale | Verification |
|---|---|---|---|---|
| **Stage labels** | Raw values | Canonical aliases | Avoid equivalent-label mismatches | Focused tests |
| **Geography** | Raw values | Canonical regions | Treat UK and United Kingdom as equivalent | Focused tests |
| **Instruments** | Exact strings | Canonical instruments | Treat convertible-instrument aliases consistently | Focused tests |
| **Tie-breaking** | Input order | Investor ID | Remove source-order dependence | Reverse-order test |
| **Weights** | Existing weights | Unchanged | Keep the normalisation effect separate | Config comparison |
| **Semantic mode** | Mixed production dependency | Explicit selectable mode | Make repeated runs comparable | Mode checks |
| **Output fields** | Existing contract | Unchanged | Keep the frontend compatible | Integration tests |

# 4. Evaluation Methodology

## 4.1 Study Design and Frozen Inputs

I evaluated the matcher as a paired engineering case study using four real funding rounds, reported as Cases 1 to 4 because the records are private. Startup-supplied fields kept the application's input shape, and blank values remained blank. I did not replace missing ticket or instrument fields with zero because that would add information the founder never supplied. The cases came from platform work available to the project. They are useful operational tests, but not a random sample of ClimateTech fundraising.

The candidate pool came from four saved Replit runs dated 12 August 2026. Each file held the same 383 investor objects and IDs. The saved data reproduced the 40 platform Top-10 IDs, ranks and scores, all 80 rows in the platform review sheet, and five rule-score columns for 4 x 383 candidates. I used this as the historical snapshot behind the live runs. The files also showed a gpt-4o-mini configuration with 60 semantic candidates and no feedback context. I could not recover the deployment commit, original Airtable order or byte-for-byte GPT responses, so the platform execution remains only partly reproducible.

## 4.2 Compared Profiles and Controls

Both profiles were rerun with the same four inputs and the same verified snapshot of 383 investors. I fixed Top-K at 10, kept the production weights, removed historical feedback and used the same investor-ID tie-break on both sides. The semantic mode was also held constant as deterministic local text similarity. This point matters. Using a local score for one profile and GPT for the other would mix the effect of normalisation with a change in semantic method. Under the frozen setup, legacy reads raw stage, geography and instrument labels, while normalisation-v1 maps known aliases before applying the same rules.

Each run stored hashes for its inputs, candidate pool, matcher file and configuration. The evaluator recorded the Git commit and rejected incomplete review files. These controls cannot make the original platform run deterministic, but they do make the local paired comparison repeatable. The project had mixed up two questions: whether normalisation helps when the setup is fixed, and whether a new profile exactly reproduces an old GPT-backed run. The main comparison answers only the first.

## 4.3 Pooled Algorithm-blinded Qualification

For each case, I pooled the Top 10 from both profiles and removed duplicates by investor ID. Across the four cases this produced 56 unique case-investor pairs. A deterministic shuffle created the reviewer CSV. The profile name, rank, score and configuration were kept out of that file and stored in a private manifest instead. The reviewer could not tell whether a row came from normalisation-v1 or whether it had originally appeared at rank one. The qualification question asked whether the investor would plausibly accept a first meeting after cold contact from the startup at the time of review.

One Claude Opus 5 model-assisted pass produced the judgements, using a binary up/down label and a required explanation. These were not human expert decisions. I transferred 39 pairs from the earlier frozen review by exact case and investor ID, then sent the remaining 17 through the same protected format. All 56 rows were complete, with 16 positive and 40 negative labels. Because the whole pool used one reviewer ID, there is no estimate of agreement between reviewers.

## 4.4 Metrics, Integrity and Reporting Boundary

For a cut-off k, precision is the number of positive labels in the first k ranks divided by k. I calculated P@5 and P@10 for every case and then took the unweighted mean across the four cases. Each paired delta is normalisation-v1 minus legacy. I do not report a p-value, confidence interval or population effect size. Four selected cases and one model-assisted label set cannot support that inference. The results answer a smaller descriptive question about these frozen lists.

The returned 17-row file included every requested ID once, with valid labels and comments. Protected evidence cells were unchanged. An independent calculation produced the same case and macro metrics as the formal evaluator. Chuan Ju then checked the private package on a separate machine, confirmed its SHA-256 and ran the offline verifier. It returned PASS_FINAL with 56/56 coverage and no discrepancies. This verifies the package and arithmetic, not the relevance judgements. The report uses anonymised aggregates. Startup names, investor comments and the private snapshot remain outside the report and repository. Figure 2 shows the route from frozen inputs to the final metrics.

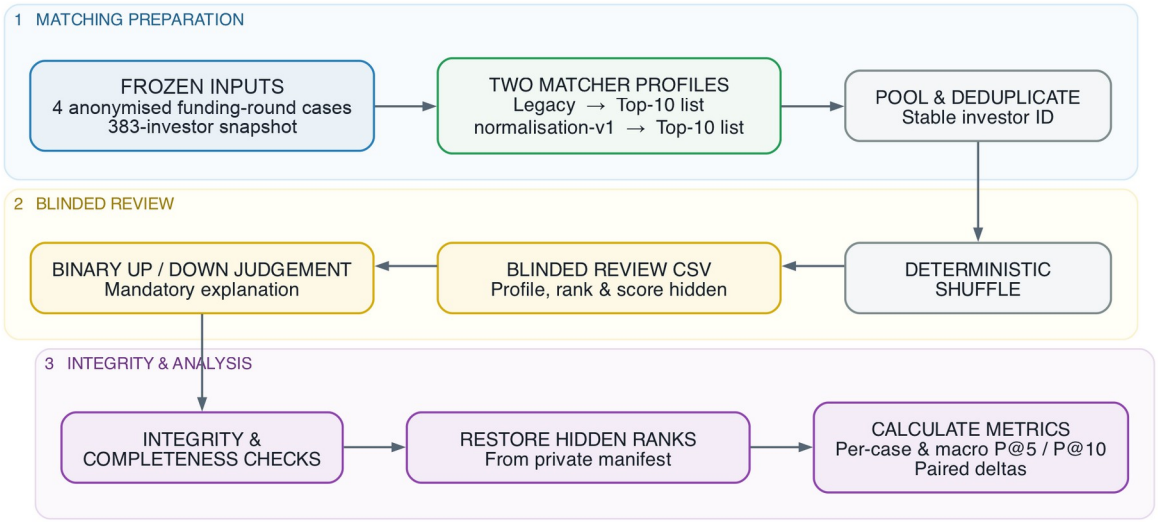



Figure 2. Paired pooled algorithm-blinded evaluation workflow.

Table 2. Design, controls and evidence status of the four-case authoritative snapshot comparison.

| Item | Final specification | Evidence status |
|---|---|---|
| **Funding-round cases** | Four real cases, reported anonymously | Complete |
| **Candidate pool** | Verified historical snapshot, 383 investors | Frozen and checked by receiver |
| **Compared profiles** | legacy versus normalisation-v1 | Complete |
| **Semantic mode** | Deterministic local text similarity | Held constant |
| **Feedback** | None | Held constant |
| **Top-K** | 10 per profile and case | Fixed |
| **Unique pooled pairs** | 56 | 56/56 complete |
| **Review type** | One Claude Opus 5 model-assisted pass | Not a human expert review |
| **Labels** | Binary up/down plus mandatory explanation | 56/56 complete |
| **Primary metrics** | P@5 and P@10 | Independently recomputed |
| **Historical platform** | Four saved gpt-4o-mini runs | Outputs verified; full replay unavailable |
| **Receiver verification** | Frozen private package | PASS_FINAL |

# 5. Results

## 5.1 Implementation and Evidence Checks

The offline readiness path passed on fictional fixtures while running the real matcher through both profiles. Focused checks covered stage, geography and instrument aliases, reverse-order ties, semantic-mode boundaries, invalid configurations, duplicate investor IDs and pooled-review integrity. The standalone interface also ran both profiles without Airtable or an API key. At the freeze point, the matcher source was byte-identical in my repository and the supervisor repository. This established that the implementation was ready for evaluation. It did not show that the ranking was better.

The saved-run audit addressed a different problem. It found one consistent historical snapshot of 383 investors, reproduced the rule scores and recovered the production semantic configuration. I still could not regenerate the GPT-backed Top 10 exactly because the original input order and model responses were missing. For that reason, I used the saved runs to establish provenance and recover the candidate pool, not as another treatment arm. The comparison below comes from the controlled local rerun.

## 5.2 Headline Comparison

Across four cases, macro P@5 rose from 0.350 for legacy to 0.500 for normalisation-v1, an absolute increase of 0.150. Macro P@10 rose from 0.300 to 0.400, an increase of 0.100. The optimised profile returned four positive matches per Top 10 on average instead of three. Relative changes were 42.9% at P@5 and 33.3% at P@10, but absolute changes are more honest for four cases. Independent recomputation and the receiver-side check reproduced the values exactly.

The average improved, but the pattern was uneven. Cases 1 and 2 improved at both cut-offs, while Cases 3 and 4 stayed the same. No case became worse. With only four cases, the mean can make an effect concentrated in two cases look more general than it is. I read the result as an initial improvement under the tested conditions, not evidence that normalisation will help every funding round.

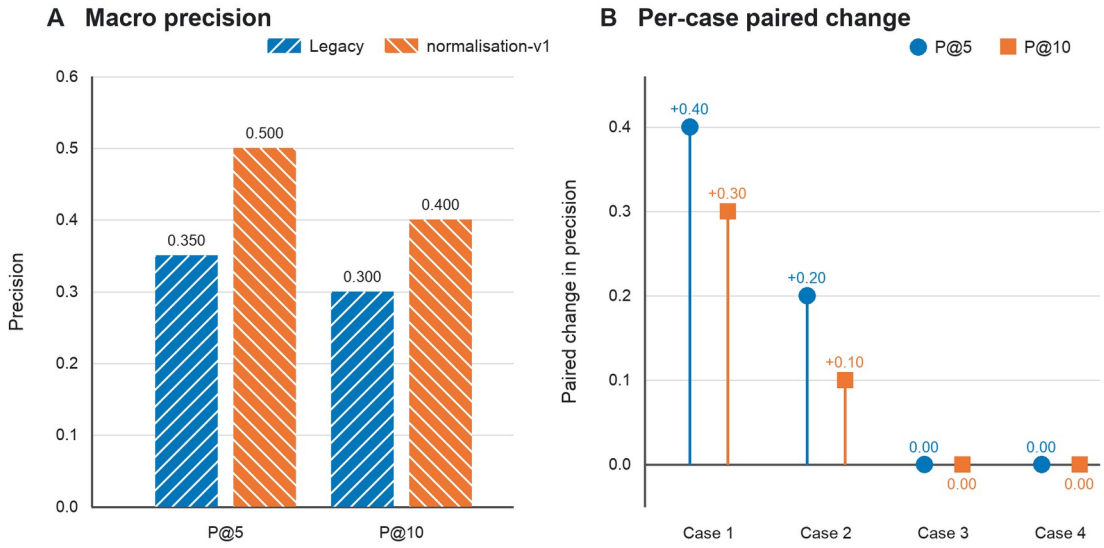



Figure 3. Macro precision and per-case paired changes in the controlled four-case comparison. Positive values indicate higher precision under normalisation-v1; results are descriptive.

## 5.3 Per-case Behaviour

Case 1 changed most: P@5 increased by 0.400 and P@10 by 0.300. Case 2 improved by 0.200 and 0.100. Cases 3 and 4 tied at both cut-offs. Their Top-10 overlaps, in case order, were 5, 6, 8 and 5 investors. Cases 1 and 4 had the largest list changes, but Case 4 gained no positive match. A different ordering is not necessarily better, and overlap does not measure relevance.

The results fit the intended mechanism: canonical labels can change rule eligibility and move candidates to the top of a tied score band. I cannot separate the effects of stage, geography and instrument mapping because the experiment did not test each normaliser alone. Portfolio conflicts, lead-investor availability, affiliation rules and missing ticket data also remain outside the change. Normalisation can repair equivalent labels. It cannot fill a field that was never collected.

Table 3. Per-case and macro precision for the legacy and normalisation-v1 profiles.

| Case | Legacy P@5 | Normalisation P@5 | Delta P@5 | Legacy P@10 | Normalisation P@10 | Delta P@10 |
|---|---|---|---|---|---|---|
| **Case 1** | 0.200 | 0.600 | +0.400 | 0.300 | 0.600 | +0.300 |
| **Case 2** | 0.400 | 0.600 | +0.200 | 0.300 | 0.400 | +0.100 |
| **Case 3** | 0.400 | 0.400 | 0.000 | 0.400 | 0.400 | 0.000 |
| **Case 4** | 0.400 | 0.400 | 0.000 | 0.200 | 0.200 | 0.000 |
| **Macro** | 0.350 | 0.500 | +0.150 | 0.300 | 0.400 | +0.100 |

## 5.4 Supplementary Pilot and Platform Provenance

An earlier pilot used a separate snapshot of 354 investors. With local text similarity and no feedback, macro P@5 moved from 0.450 to 0.550 and P@10 from 0.300 to 0.425. The direction matches the final comparison, but the size does not. I treat the pilot as a sensitivity check because its candidate pool was not the verified 12 August snapshot.

The original Replit outputs had lower qualification precision than the local optimised lists. That gap is not an estimate of the normalisation effect because the environments were different. The platform used GPT semantic scores and historical source order; the controlled comparison used deterministic text similarity and investor-ID ties. I now know that the investor snapshot was the same, but the semantic execution was not. The paired result on 383 investors is therefore the headline evidence, while the saved platform output is used only to document provenance.

# 6. Discussion and Limitations

## 6.1 Answer to the Research Question

For these four rounds and the verified 383-investor snapshot, normalisation-v1 placed more positively qualified investors near the top than legacy. Macro P@5 improved by 0.150 and P@10 by 0.100, so the research question has a positive answer within this experiment. I would not extend it to all ClimateTech startups. Two cases improved and two tied. This is a useful first test, not a settled benchmark.

Both profiles used the same text scorer, weights, candidate pool and tie-break. The observed difference is therefore consistent with cleaning stage, geography and instrument labels before rule scoring. It is still only a combined effect. Without separate ablations, I cannot tell which mapping family mattered most in each case. That is a clear next experiment. The outcome measured here is also qualification precision rather than real engagement. A positive label means the pairing passed the review question; it does not show that an introduction happened, a meeting was accepted or an investment followed.

## 6.2 What Worked and What was Difficult

Keeping legacy available was the most useful engineering choice. The baseline stayed fixed while I added the new helpers, which made failures easier to trace. The protected manifest also allowed the 17 returned rows to be merged without changing source evidence, and the metrics could be recalculated from scratch. These changes are less eye-catching than a new model, but they made the result easier to trust and audit.

Reconstructing the old production context was harder. My early local runs contained all 40 platform investors but recovered little of the platform Top 10. The saved-run export explained why: GPT scores and source order had shaped the old result. Recovering the 383 investor objects fixed the snapshot gap, but the exact GPT response and deployment state were gone. I left the production route unchanged because a similar local output would not prove that I had reproduced the live system.

## 6.3 Limitations and Threats to Validity

Scale is the clearest limitation. Four available cases cannot cover the stages, sectors, geographies and round structures that Undaunted may encounter. There were also only 16 positive labels among the 56 pooled pairs, so changing one judgement can noticeably move a case metric. Reporting every case beside the macro average makes that fragility visible, but it cannot remove it.

The labels bring a second limitation. One Claude Opus 5 process applied the same qualification question and wrote an explanation for each decision, but there was no independent second reviewer and no inter-rater statistic. The exact model build, sampling settings and web state were not exposed either, so the comments cannot be regenerated byte for byte. Chuan Ju's PASS_FINAL verifies the package, protected cells and arithmetic. It does not show agreement with each investor judgement.

## 6.4 Privacy, Ethics and Deployment Boundary

The funding rounds came from real startup material, and the investor snapshot identifies organisations. I report only case numbers and aggregate metrics. Row-level records, comments and source attachments remain outside Git. The labels evaluate a ranking system and are not investment advice. Production deployment is a separate decision. It needs authorised human review on more cases and a fresh platform snapshot, even though the guarded runtime already preserves the frontend contract.

Table 4. Threats to validity, mitigations and residual limitations.

| Threat | Affected claim | Current mitigation | Residual limitation |
|---|---|---|---|
| **Four selected cases** | General performance | Report every case beside the macro mean | Cannot generalise to the wider startup population |
| **One model-assisted reviewer** | Relevance validity | Fixed question and mandatory explanations | No human expert validation or inter-rater reliability |
| **Historical 383-investor snapshot** | Current-platform relevance | Snapshot frozen and independently verified | The live investor table can change |
| **Text-similarity test mode** | Equivalence to production GPT | Both profiles use the same semantic mode | Does not reproduce GPT behaviour |
| **Historical replay gaps** | Platform reproducibility | Saved outputs, configuration and rule scores verified | Exact deployment SHA, input order and GPT replay unavailable |
| **Mutable qualification evidence** | Label reproducibility | Procedure and member-file hashes preserved | Exact Opus build, sampling and web state unavailable |
| **Privacy boundary** | Public audit detail | Cases anonymised and private package checked | Identifiable row-level evidence cannot be published |

# 7. Conclusion and Future Work

I tested whether a small, explainable change could improve the top results from an existing ClimateTech investor matcher. I preserved the legacy path, normalised common stage, geography and instrument labels, and made experimental ranking deterministic. I also built a protected evaluation process without changing the frontend result shape. Across four anonymised cases and the verified 383-investor snapshot, macro P@5 rose from 0.350 to 0.500 and macro P@10 from 0.300 to 0.400. Two cases improved and two tied. The evidence is descriptive and model-assisted, but the metrics and package can be reproduced. The next step is a larger blinded review with at least two human reviewers and a wider range of rounds. If the result holds, normalisation-v1 should be tested on a current authorised snapshot and followed prospectively through introductions and accepted meetings. That would show whether the local gain survives real use.

# Research Declarations

## Data and code availability

The public repository contains the matching code, documentation, automated checks and fictional fixtures. The real funding-round inputs, investor snapshots, review rows and comments remain private. This report uses anonymised aggregate evidence from those files.

## Author contributions

I completed the baseline mapping, matcher changes, comparison tools, integration checks, analysis and report writing. Chuan Ju contributed to qualification and evaluation and carried out the independent receiver-side package check. Maiko Schaffrath and César A. Quilodrán-Casas supervised and reviewed the project.

## Ethics and intended use

The evaluation uses startup-supplied project data within an anonymised reporting boundary. No investor was contacted, and the labels are not investment advice. I report no conflict of interest in the comparison.

# References

Burges, C., Shaked, T., Renshaw, E., Lazier, A., Deeds, M., Hamilton, N. and Hullender, G. (2005) 'Learning to Rank using Gradient Descent', Proceedings of the 22nd International Conference on Machine Learning, pp. 89-96. https://doi.org/10.1145/1102351.1102363.

Järvelin, K. and Kekäläinen, J. (2002) 'Cumulated gain-based evaluation of IR techniques', ACM Transactions on Information Systems, 20(4), pp. 422-446. https://doi.org/10.1145/582415.582418.

Liu, T.-Y. (2009) 'Learning to Rank for Information Retrieval', Foundations and Trends in Information Retrieval, 3(3), pp. 225-331. https://doi.org/10.1561/1500000016.

Manning, C.D., Raghavan, P. and Schütze, H. (2008) Introduction to Information Retrieval. Cambridge: Cambridge University Press.

Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks', Proceedings of EMNLP-IJCNLP 2019, pp. 3982-3992. https://doi.org/10.18653/v1/D19-1410.